Loop Gestión: Plataforma Académica Móvil Full-Stack con Automatización de Notificaciones y Asistente de Inteligencia Artificial

Sebastián Grande Moreno, Juan Carlos Díaz Cifuentes
Facultad de Ingeniería, Universidad de San Buenaventura, Bogotá, Colombia
Optativa II (10348): Desarrollo de Aplicaciones para Dispositivos Móviles · Docente: MGTR. Ing. Jairo Armando Salcedo Aranda · Noviembre de 2026

***Resumen*—Este artículo presenta Loop Gestión, una plataforma académica compuesta por una aplicación móvil nativa en Android (Kotlin) y una API REST en Node.js/Express con base de datos MySQL ejecutada en Docker. La plataforma centraliza las actividades, entregas y calificaciones de cada materia para dos roles, estudiante y docente, autenticados con Firebase Authentication. Al publicar una actividad, la API dispara un webhook de Make.com que notifica a los estudiantes por Gmail y registra el evento en Google Sheets. Un asistente conversacional, LoopBot IA, informa al estudiante sus pendientes y notas. El proyecto se gestionó con Scrum en Azure DevOps en tres sprints, con seis elementos de backlog que suman 58 puntos de esfuerzo, y cuenta con un pipeline de integración continua en Azure Pipelines. Se describen la arquitectura, el modelo de datos, las integraciones, los controles de seguridad y la estrategia de pruebas.**

***Palabras clave*—Android, Kotlin, Node.js, MySQL, Docker, Firebase, Make.com, inteligencia artificial, Scrum, Azure DevOps.**

I. INTRODUCCIÓN

En un curso universitario, la información de actividades, fechas de entrega, archivos y calificaciones suele estar dispersa entre correos, grupos de mensajería y hojas de cálculo. El estudiante pierde plazos y no conoce su situación académica con facilidad; el docente dedica tiempo a recordar fechas y consolidar notas.

Loop Gestión responde a la pregunta: *¿cómo puede una aplicación móvil centralizar actividades, entregas y calificaciones por materia, notificar automáticamente a los estudiantes y ayudarles a organizarse con un asistente inteligente?* La sección II describe la arquitectura; la III, el modelo de datos y la API; la IV, las integraciones y LoopBot; la V, la seguridad; la VI, la metodología; la VII, el estado del proyecto y las pruebas; y la VIII, las conclusiones.

II. ARQUITECTURA

El sistema sigue una arquitectura **cliente-servidor por capas** (Fig. 1). La app Android presenta las pantallas de cada rol y se comunica mediante Retrofit con una API REST en Node.js/Express, que concentra las reglas de negocio y es el único componente con acceso a la base de datos MySQL. MySQL se ejecuta en un contenedor Docker para que el entorno sea idéntico en todos los equipos.

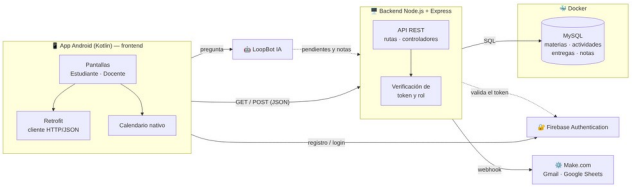


Fig. 1. Arquitectura de Loop Gestión: app Android, API Node.js, MySQL en Docker y servicios externos.

TABLA I

TECNOLOGÍAS DE LA SOLUCIÓN

Componente	Tecnología
App móvil	Android nativo · Kotlin
Cliente HTTP	Retrofit (JSON)
API	Node.js + Express
Base de datos	MySQL en Docker
Autenticación	Firebase Authentication
Automatización	Make.com · Gmail · Google Sheets
Asistente	LoopBot IA
CI/CD y gestión	Azure Pipelines · Azure Boards

III. MODELO DE DATOS Y API

El dominio se modela con cinco entidades: **Usuario** (enlazado con Firebase por su uid y con un rol), **Materia**, **Matrícula** (relación estudiante–materia), **Actividad** (fecha límite y valor porcentual) y **Entrega** (archivo, comentario, nota de 0,0 a 5,0 y retroalimentación). La matrícula garantiza que cada estudiante solo vea las actividades de sus materias.

La API expone recursos REST: lecturas con **GET** (materias, actividades, boletín) y escrituras con **POST** y **PUT** (crear actividad, enviar entrega, calificar). Cada petición incluye el *ID token* de Firebase; un *middleware* lo verifica y comprueba el rol antes de ejecutar la operación, de modo que un estudiante no puede, por ejemplo, calificar entregas aunque modifique la app.

IV. INTEGRACIONES Y ASISTENTE

A. Notificaciones con Make.com

Cuando el docente publica una actividad con la opción «Notificar a los estudiantes», la API envía un POST al webhook de un escenario de Make.com. El escenario envía un correo por Gmail a los estudiantes matriculados y agrega una fila en Google Sheets como registro. Esta integración evita programar el envío de correos y la API de hojas de cálculo en el backend.

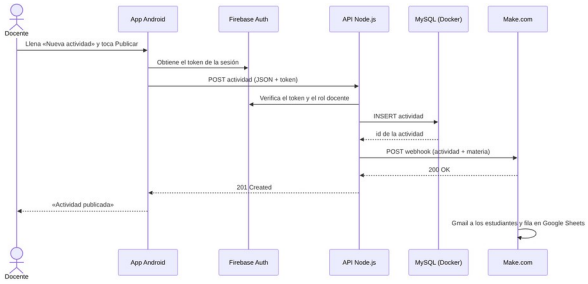


Fig. 2. Secuencia al publicar una actividad: validación del token, registro en MySQL y notificación por Make.com.

B. LoopBot IA

LoopBot es un asistente conversacional para el estudiante: responde qué actividades tiene pendientes, cómo va en sus notas, explica temas y propone cómo organizar la semana. La pantalla advierte que es un sistema de IA que puede equivocarse. Su comportamiento cuenta con pruebas automatizadas en *tests/chatbot.test.js*, integradas al repositorio mediante el *pull request* «Integración de pruebas de LoopBot IA y estructura del repositorio».

V. SEGURIDAD

Los controles se organizaron con OWASP MASVS [3] y OWASP API Security Top 10 [4]: autenticación delegada en Firebase, autorización por rol en la API, base de datos no expuesta (solo accesible desde la API), consultas parametrizadas contra inyección SQL, validación de entradas en la app y en el servidor, y credenciales (base de datos, cuenta de servicio de Firebase, webhook de Make.com y clave de IA) en variables de entorno fuera del repositorio. La verificación combina análisis **estático** (detect-secrets, mobsfscan, npm audit, Android Lint) y **dinámico** (peticiones sin token, con rol incorrecto o con datos maliciosos contra la API en ejecución).

VI. METODOLOGÍA

El proyecto se gestionó con Scrum [5] en Azure DevOps. El Product Backlog contiene seis elementos (PBI) distribuidos en tres sprints de dos semanas (Tabla II). El repositorio en Azure Repos se organiza en *docs/*, *src/* y *tests/*, con ramas *main*, *develop* y ramas *feature*. El tablero Scrum y un dashboard con seis widgets permiten seguir el avance.

TABLA II

PRODUCT BACKLOG Y SPRINTS

PBI	Contenido	Esf.	Sprint
1	Arquitectura full-stack, MySQL en Docker, API Node.js	8	1
2	Autenticación multi-rol en Firebase y matrícula	8	1
3	Gestión docente: actividades, calendario, Retrofit	13	2
4	Portal del estudiante: entregas, KPIs, boletín	13	2
5	Automatización con Make.com	8	3
6	LoopBot IA y pipeline CI/CD en Azure	8	3

Sprint 1 (7–20 oct): análisis, requisitos, diseño y autenticación. Sprint 2 (21 oct–3 nov): funcionalidades principales. Sprint 3 (4–17 nov): Make.com, chatbot, pruebas y despliegue.

VII. ESTADO DEL PROYECTO Y PRUEBAS

Al corte del 4 de noviembre de 2026, el tablero mostraba los PBI 1, 2 y 6 en estado *Done*, los PBI 3 y 4 en *Committed* y el PBI 5 en *Approved*; el gráfico de *burnup* del dashboard reportaba 50 % del alcance completado. La ejecución #20261104.1 del pipeline sobre la rama *develop* terminó satisfactoriamente en 43 s, con tres advertencias de mantenimiento: la migración de la imagen *ubuntu-latest* y el reemplazo de la tarea deprecada *NodeTool@0* por *UseNode@1*.

El plan de pruebas (IEEE 829 [2]) define 32 casos que cubren registro y autenticación por rol, flujos de estudiante y docente, notificaciones, LoopBot, autorización de la API, despliegue con Docker y el pipeline. Sus resultados se registran en la matriz de casos de prueba a medida que se ejecutan en el emulador y contra la API.

VIII. CONCLUSIONES

Loop Gestión integra en una sola plataforma móvil la gestión académica de estudiantes y docentes. Separar la app de la base de datos mediante una API con autorización por rol protege la información y concentra las reglas de negocio; Docker hace reproducible el entorno; y Make.com permite automatizar notificaciones sin aumentar la complejidad del backend. Azure DevOps aportó trazabilidad entre backlog, código y ejecuciones del pipeline.

Trabajo futuro. (1) Publicar la API con HTTPS en un servicio en la nube; (2) notificaciones *push* con Firebase Cloud Messaging; (3) panel de analítica para el docente; y (4) integración con el sistema académico institucional.

REFERENCIAS

[1] *Systems and software engineering — Requirements engineering*, ISO/IEC/IEEE 29148:2018.

[2] *IEEE Standard for Software and System Test Documentation*, IEEE Std 829-2008.

[3] OWASP Foundation, *Mobile Application Security Verification Standard (MASVS)*, v2.0, 2023.

[4] OWASP Foundation, *OWASP API Security Top 10*, 2023. [En línea]. Disponible: https://owasp.org/API-Security/

[5] K. Schwaber y J. Sutherland, *La Guía de Scrum*, 2020. [En línea]. Disponible: https://scrumguides.org

[6] Google, "Guide to app architecture," *Android Developers*. [En línea]. Disponible: https://developer.android.com/topic/architecture

[7] Google, "Verify ID tokens," *Firebase Documentation*. [En línea]. Disponible: https://firebase.google.com/docs/auth/admin/verify-id-tokens

[8] Microsoft, "Azure Pipelines documentation," *Microsoft Learn*. [En línea]. Disponible: https://learn.microsoft.com/azure/devops/pipelines/